

1. Bubble sort is a popular sorting algorithm. It works by repeatedly swapping adjacent elements that are out of order.

BUBBLE-SORT(A)

```

1 for  $i = 1$  to  $\text{length}[A] - 1$ 
2   for  $j = \text{length}[A]$  downto  $i + 1$ 
3     if  $A[j] < A[j - 1]$ 
4       exchange  $A[j] \leftrightarrow A[j - 1]$ 

```

(a) Illustrate the operation of BUBBLE-SORT on the array $A = (37, 24, 52, 27, 48, 26)$.

(b) What is the worst-case running time of BUBBLE-SORT? How does it compare to the running time of INSERTION-SORT?

(c) Use loop invariants to prove the correctness of the algorithm.

(a)

排序的 $\leftarrow$

(1st) $i=1$ 24 | 37 > 26 52 > 27 48

(1st) $i=2$ 24 > 26 | 37 > 27 52 48

(1st) $i=3$ 24 > 26 > 27 | 37 48 52

(1st) $\begin{cases} i=4 & 24 > 26 > 27 > 37 | 48 > 52 \\ i=5 & 24 > 26 > 27 > 37 > 48 | 52 \end{cases}$

(b) Bubble sort 中:

(1st) $i=1, j=n-2 \Rightarrow$ 跑 $n-1$ 次

$i=2, j=n-3 \Rightarrow$ 跑 $n-2$ 次

$\vdots$

$i=n-1, j=n-n \Rightarrow$ 跑 1 次

$\Rightarrow$ 共跑 $1+2+\dots+n-1$ 次 $\Rightarrow O(n^2)$

不論 best-case or worst-case

皆會跑完 $\Rightarrow$ for-loop

所花時間皆 $O(n^2)$ (可不寫出 $O(n^2)$)

(1st)

Insertion sort 的 worst-case 時:

每次都要交換, 直到陣列頭

$i=2, j=2-1 \Rightarrow$ 跑 2 次

$i=3, j=3-1 \Rightarrow$ 跑 3 次

$\vdots$

$i=n, j=n-1 \Rightarrow$ 跑 n 次

$\Rightarrow$ 共跑 $2+3+\dots+n$ 次 $\Rightarrow O(n^2)$

在 worst-case 時的複雜度與 Bubble sort 一樣

(c)

(1st) loop invariant: 執行第 i 個 loop 前, 前 $i-1$ 位數從小到大已排序好, 且皆小於等於第 $i \sim n$ 位數

(1st) 初始條件: $i=1$, loop 開始前, 沒有數字是排序好的 $\Rightarrow$ 成立

(1st) 保持條件: 設 $i=k$

垂直行 loop 前, 第 $1 \sim k-1$ 位數為已排序好的且皆小於等於第 $k \sim n$ 位數

垂直行 loop 後, 會將第 $k \sim n$ 位數中最小的數放在第 k 位

$\Rightarrow$ 第 $1 \sim k$ 位數為小到大排序好的且皆小於等於第 $k+1 \sim n$ 位數

$\Rightarrow$ 正確維護性質

(1st) 終止條件: 當 loop 結束後 $i=n$, 所以根據 loop invariant, 前 $n-1$ 位數從小到大已排序好, 且皆小於等於第 n 位數, 所以 $A[1] \sim A[n]$ 為小到大已排序好的數列

1. Consider sorting n numbers stored in array A by first finding the smallest element of A and exchanging it with the element in $A[1]$. Then find the second smallest element of A , and exchange it with $A[2]$. Continue in this manner for the first $n-1$ elements of A .

SELECTION-SORT(A)

```

1 for  $i = 1$  to  $\text{length}[A] - 1$ 
2   let  $\text{smallest} \leftarrow i$ 
3   for  $j = i + 1$  to  $\text{length}[A]$ 
4     if  $A[j] < A[\text{smallest}]$ 
5       let  $\text{smallest} \leftarrow j$ 
6   exchange  $A[i] \leftrightarrow A[\text{smallest}]$ 

```

- (a) Illustrate the operation of SELECTION-SORT on the array $A = \langle 37, 24, 52, 27, 48, 26 \rangle$.
 (b) Give the best-case and worst-case running times of SELECTION-SORT in Θ -notation.
 (c) Use loop invariants to prove the correctness of the algorithm.

1. (a) $A = \langle 37, 24, 52, 27, 48, 26 \rangle$

3分 step 1. $A = \langle 24, 37, 52, 27, 48, 26 \rangle$

2. $A = \langle 24, 26, 52, 27, 48, 37 \rangle$

3. $A = \langle 24, 26, 27, 52, 48, 37 \rangle$

4. $A = \langle 24, 26, 27, 37, 48, 52 \rangle$

5. $A = \langle 24, 26, 27, 37, 48, 52 \rangle$

3分

(b) 無論 array 如何排列, 都須跑完 n 層 loop

best case = worst case = $\Theta(n^2)$

(c) loop invariant: 執行第 i 個 loop 前, 前 $i-1$ 位數從小到大排好, 且皆小於等於第 $i \sim n$ 位數

1分

1分 初始條件: $i=1$, loop 開始前, 沒有數字排好 $\Rightarrow$ 成立

1分 保持條件: 設 $1 \leq k$

loop 執行前, 第 $1 \sim k-1$ 位數已由小至大排好, 且皆小於等於第 $k \sim n$ 位數

loop 執行後, 將第 $k \sim n$ 位數中之最小值放在第 k 位

$\therefore$ 第 $1 \sim k$ 位數已由小至大排好, 且小於等於第 $k+1 \sim n$ 位數

$\Rightarrow$ 正確維護性質

1分 終止條件: 當 loop 結束後, $i=n$, 根據 loop invariant, 前 $n-1$ 位數已由小至大排好

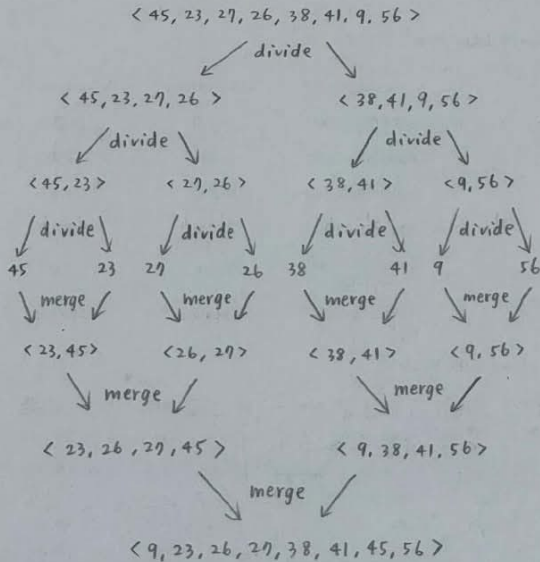
且皆小於等於第 n 位數, $\therefore A[1] \sim A[n]$ 已由小至大排好

1. Consider the Merge-Sort:

- Illustrate the operation of MERGE-SORT on the array $A = (45, 23, 27, 26, 38, 41, 9, 56)$.
- What is the best-case running time of MERGE-SORT?
- What is the worst-case running time of MERGE-SORT?

1.

(a)



(b)(c)

無論 worst-case 或是 best-case 皆需進行 divide, conquer 和 combine
 設 n 為 array 的大小, h 為遞迴層數

	共 h 層				
第 i 層	0	1	2	...	$h-1$
array size	n	$\frac{n}{2}$	$\frac{n}{4}$	...	$\frac{n}{2^{h-1}}$

$$\frac{n}{2^{h-1}} = 1 \Rightarrow n = 2^{h-1} \Rightarrow h = \lg n + 1$$

$$\text{total cost} = cn(\lg n + 1)$$

$$= cn \lg n + cn$$

$$= \Theta(n \lg n)$$

$$\text{worst-case} = \text{best-case} = \Theta(n \lg n) *$$

2. Consider the searching problem:

Input: A sequence of n numbers $A = \langle a_1, a_2, \dots, a_n \rangle$ and a value v .

Output: An index i such that $v = A[i]$ or the special value NIL if v does not appear in A .

Write pseudocode for *linear search*, which scans through the sequence, looking for v . Using a loop invariant, prove that your algorithm is correct. Make sure that your loop invariant fulfills the three necessary properties.

2. Linear-search (A, v)

```
1. for  $i \leftarrow 1$  to  $n$ 
2.   if  $A[i] == v$ 
3.     return  $i$ 
4. return NIL
```

證明:

Loop invariant: 執行第 i 個 loop 前, $A[1] \sim A[i-1]$ 都不等於 v

Initialization: $i=1$, loop 開始前, A 尚未被檢查

Maintenance: 設 $i=k$,

loop 開始前, $A[1] \sim A[k-1]$ 皆不等於 v

loop 開始後, 若 $A[k]$ 等於 v , 則程式

立刻結束並回傳 index k , 若不相等,

則進入下一個 loop, 且 $A[1] \sim A[k]$

都不等於 $v \Rightarrow$ loop invariant 被維持。

Termination: 程式結束時, i 分 2 種情況。

情形 1: $i \neq n+1$, 則 $v = A[i]$, 找到 v

情形 2: $i = n+1$, 根據 loop invariant,

$A[1] \sim A[n]$ 都不等於 v ,

因此 v 不存在。

程式的執行, 在以上 2 種情況下皆正確。

1.

4 ← 6 j

(a) (37, 24, 52, 27, 48, 26) (len=6)

完成排序

i=1 24 37 26 52 27 48

i=2 24 26 37 48 52 27

i=3 24 26 27 37 48 52

i=4 24 26 27 37 48 52

i=5 24 26 27 37 48 52

(b) Bubble Sort, Insertion Sort, Selection Sort, Merge Sort 的 worst case 比較。

Bubble Sort:

i=1, j=n-1 → 比較 n-1 次

i=2, j=n-2 → 比較 n-2 次

⋮

i=n-1, j=n → 比較 1 次

總次數: $\frac{n(n-1)}{2} \Rightarrow O(n^2)$

Worst Case: $O(n^2)$

Insertion Sort:

對於 Insertion Sort, 最壞情況是每次都要交換:

i=2, j=2~1, 換 1 次

i=3, j=3~1, 換 2 次

⋮

i=n, j=n~1, 換 n-1 次

總次數: $\frac{n(n-1)}{2} \Rightarrow O(n^2)$

Worst Case: $O(n^2)$

Selection Sort:

對於 Selection Sort, 最壞情況是總共要掃描 $n-1$ 輪來找最小值

$i=1, j=2 \sim n$

$i=2, j=3 \sim n$

$\vdots$

$i=n-1, j=n \sim n$

總次數:

$$\Rightarrow \frac{n(n-1)}{2} \Rightarrow O(n^2)$$

Merge Sort:

Merge Sort 使用分治法

每次都會將長度 n 的陣列拆半, 直至長度為 1。

每次合併都要處理 n 個元素, 耗費 $O(n)$ 時間。

$$T(n) = 2T(n/2) + cn$$

$$\rightarrow O(n \log n)$$

(C) Use loop-invariant to prove bubble sort

loop invariant: 當執行第 i 個 loop 之前, 陣列第 $1 \sim i-1$ 個元素皆已照順序排好, 且包含全陣列最小的 $i-1$ 個元素。

初始條件: 當 $i=1$ 時, 子陣列僅有一個元素, 一個元素保證排序好, 不變性成立。

保持條件: 每次內迴圈執行時, 都會將陣列 $i \sim n$ 中的最小值交換到 i 的位置, 確保下一次 $i+1$ 時, $1 \sim i$ 個元素都被排序。

終止條件: 當 $i=n$ 時, loop 停止, 根據不變性, 陣列 $1 \sim n$ 已包含 $n-1$ 個最小元素, 且照順序排好。

$\therefore$ 前 $n-1$ 個元素已排好, 最後剩的 $A[n]$ 必為最大值。

1. Consider sorting n numbers stored in array A by first finding the smallest element of A and exchanging it with the element in $A[1]$. Then find the second smallest element of A , and exchange it with $A[2]$. Continue in this manner for the first $n - 1$ elements of A .

SELECTION-SORT(A)

```
1 for  $i = 1$  to  $\text{length}[A] - 1$ 
2   let  $\text{smallest} \leftarrow i$ 
3   for  $j = i + 1$  to  $\text{length}[A]$ 
4     if  $A[j] < A[\text{smallest}]$ 
5       let  $\text{smallest} \leftarrow j$ 
6   exchange  $A[i] \leftrightarrow A[\text{smallest}]$ 
```

- (a) Illustrate the operation of SELECTION-SORT on the array $A = (37, 24, 52, 27, 48, 26)$.
(b) Give the best-case and worst-case running times of SELECTION-SORT in Θ -notation.
(c) Use loop invariants to prove the correctness of the algorithm.

(a) 37, 24, 52, 27, 48, 26 (len = 6)

← 已排序好

$i=1$ 24 | 37 52 27 48 26

$i=2$ 24 26 | 52 27 48 37

$i=3$ 24 26 27 | 52 48 37

$i=4$ 24 26 27 37 | 48 52

$i=5$ 24 26 27 37 48 | 52

(b) 無論陣列怎麼排列, 都需執行 2 層 for-loop

$\Rightarrow \text{best case} = \text{worst case} = \Theta(n^2)$

1. Consider sorting n numbers stored in array A by first finding the smallest element of A and exchanging it with the element in $A[1]$. Then find the second smallest element of A , and exchange it with $A[2]$. Continue in this manner for the first $n - 1$ elements of A .

SELECTION-SORT(A)

```
1 for  $i = 1$  to  $\text{length}[A] - 1$ 
2   let  $\text{smallest} \leftarrow i$ 
3   for  $j = i + 1$  to  $\text{length}[A]$ 
4     if  $A[j] < A[\text{smallest}]$ 
5       let  $\text{smallest} \leftarrow j$ 
6   exchange  $A[i] \leftrightarrow A[\text{smallest}]$ 
```

- (a) Illustrate the operation of SELECTION-SORT on the array $A = (37, 24, 52, 27, 48, 26)$.
(b) Give the best-case and worst-case running times of SELECTION-SORT in Θ -notation.
(c) Use loop invariants to prove the correctness of the algorithm.

(c)

loop invariants: 執行第 i 次迴圈前, 前 $i-1$ 位都由小到大排好, 且皆小於等於 $i \sim n$ 位數。

初始條件: $i=1$, 沒有數字排序好 $\Rightarrow$ 成立

保持條件: let $i=k$, loop 執行前, $1 \sim k-1$ 都由小到大排好, 且皆小於 $k \sim n$ 位數
loop 執行後, 會將 $k \sim n$ 位中最小的數置於第 k 位

$\rightarrow$ 確保 $1 \sim k$ 都由小到大排好, 且皆小於 $k+1 \sim n$ 。

$\rightarrow$ 正確維護 loop invariants

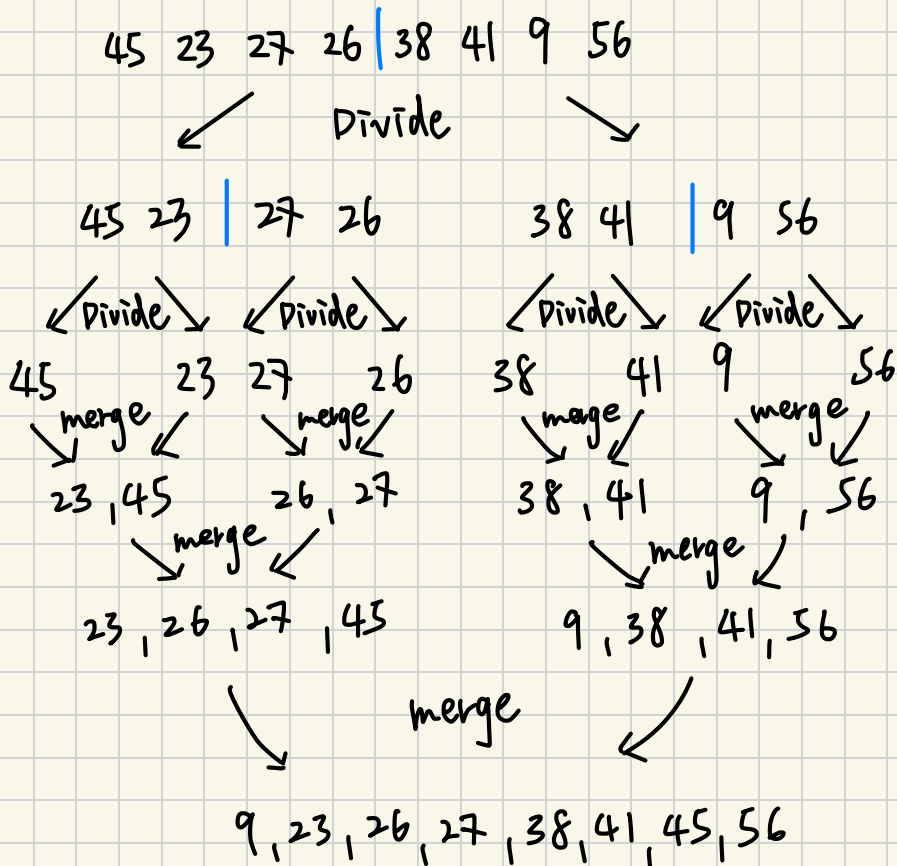
終止條件: 當 loop 終止時 $i=n$, 依據 loop invariants, 前 $1 \sim n-1$ 都由小到大排好, 且皆小於等於第 n 位數。

$\rightarrow$ 可知陣列 $1 \sim n$ 位都已由小到大排序

1. Consider the Merge-Sort:

- (a) Illustrate the operation of MERGE-SORT on the array $A = \langle 45, 23, 27, 26, 38, 41, 9, 56 \rangle$.
- (b) What is the best-case running time of MERGE-SORT?
- (c) What is the worst-case running time of MERGE-SORT?

(a) 45 23 27 26 38 41 9 56



1. Consider the Merge-Sort:

- (a) Illustrate the operation of MERGE-SORT on the array $A = (45, 23, 27, 26, 38, 41, 9, 56)$.
 (b) What is the best-case running time of MERGE-SORT?
 (c) What is the worst-case running time of MERGE-SORT?

(b) (c)

CH2 P.25

無論 best case、worst case 皆需對陣列進行 Divide 和 merge 操作。let n = 陣列大小, h = 遞迴層數

層數	0	1	2	3	...	$h-1$	已知最後一層分到剩下 1 個元素, 則
陣列大小	n	$\frac{n}{2}$	$\frac{n}{4}$	$\frac{n}{8}$	...	$\frac{n}{2^{h-1}}$	$\frac{n}{2^{h-1}} = 1, n = 2^{h-1}$
共 h 層							$h = \log_2 n + 1$

$$\text{Total Cost} = cn(\log_2 n + 1) = cn \log_2 n + cn$$

$$\text{worst case} = \text{best case} = \Theta(n \log n)$$

2. Consider the searching problem:

Input: A sequence of n numbers $A = \langle a_1, a_2, \dots, a_n \rangle$ and a value v .

Output: An index i such that $v = A[i]$ or the special value NIL if v does not appear in A .

Write pseudocode for *linear search*, which scans through the sequence, looking for v . Using a loop invariant, prove that your algorithm is correct. Make sure that your loop invariant fulfills the three necessary properties.

pseudocode:

linear-search(A, v):

for $i = 1$ to n :

if $A[i] == v$:

return i

return NIL

loop invariant: 執行第 i 次 loop 前, 數值 v 不存在於陣列 $1 \sim i-1$ 的位置。

初始條件: $i = 1$, 陣列位置 $1 \sim 0$ 為空, 顯然不存在 v 。 $\Rightarrow$ 成立

保持條件: let $i = k$

loop 開始前, $A[1] \sim A[k-1]$ 皆不等於 v 。

loop 開始後, 若 $A[k] == v$, 則程式立即終止回傳 index k ; 若不相等, 則會進入下一個 loop, $i = k+1$, 且 $A[1] \sim A[k]$ 皆不等於 v 。

$\rightarrow$ 正確維護 loop invariant

終止條件: 當 loop 結束時, 可以分成 2 種情形:

Case 1: $i \neq n+1$, 則陣列 A 中存在 v 。

Case 2: $i == n+1$, 依據 loop invariant, $A[1] \sim A[n]$ 都不等於 v , 所以 v 不存在於陣列 A 中。